

Desarrollo Web para portal de Inteligencia Analítica

Lección 2. Fundamentos de Streamlit

Introducción

¿Alguna vez has sentido que tienes muchas actividades en un día como tomar clases, hacer ejercicio o estar con tu familia, y necesitas priorizar lo importante? En el mundo de los datos pasa lo mismo. Por eso existen los *dashboards* o cuadros de mando; herramientas que permiten visualizar la priorización de actividades para su organización.

Por ejemplo, las empresas manejan toneladas de datos valiosos. Los equipos técnicos analizan la información, mientras que los equipos no técnicos necesitan verla de forma simple y organizada. Ahí es donde un *dashboard* dinámico entra en juego para ejecutar consultas o generar visualizaciones interactivas que facilitan la comprensión de la información.

El desarrollo de plataformas web está en constante evolución, influenciado por tendencias tecnológicas, cambios en el comportamiento de los consumidores y mejoras en la experiencia de usuario.



Actualmente, el cómputo en la nube ha revolucionado la forma de desarrollar e implementar aplicaciones web. Tecnologías como **AWS**, **Azure** y **Google Cloud** permiten crear aplicaciones escalables y flexibles sin invertir en infraestructura física, utilizando desde **HTML** o una combinación con **CSS**, hasta **JavaScript**, el lenguaje de programación más popular.

Además, la comunidad ha creado *frameworks* como **Angular**, **React**, **Flutter** o **Streamlit**, que facilitan la creación de aplicaciones web.

En esta lección, estudiaremos los fundamentos y alcances de **Streamlit**. ¿Estás listo? ¡Comencemos!

Introducción a Streamlit

¿Qué es la programación orientada a eventos?

La programación orientada a eventos es un paradigma de programación donde el flujo de control del programa se determina por los eventos que ocurren durante su ejecución, como acciones del usuario o mensajes de otros programas. **En este paradigma, los eventos pueden ser acciones del usuario, como hacer clic en un botón o escribir en un campo de texto. También pueden ser acciones internas del sistema, como recibir datos de una red o terminar una descarga.**

Este enfoque separa la lógica de procesamiento de eventos del resto del código, situación que le permite a las aplicaciones responder de manera eficiente a múltiples eventos concurrentes. En ese sentido, cuando un evento es detectado, el controlador de eventos correspondiente se ejecuta para manejarlo, definiendo las acciones que deben realizarse en respuesta al evento.

La programación orientada a eventos es especialmente útil en el desarrollo de interfaces de usuario interactivas, como aplicaciones web y de escritorio, donde los eventos del usuario desencadenan acciones específicas en la interfaz, mejorando la experiencia del usuario. Además, es una pieza clave en sistemas distribuidos y redes para manejar la comunicación asíncrona, así como en aplicaciones que

requieren procesamiento de datos en tiempo real, como los sistemas de monitoreo de infraestructuras o análisis de datos.

Por su parte, **Streamlit** es una biblioteca de **Python** que facilita la creación de aplicaciones web interactivas y de datos, y se utiliza para implementar la programación orientada a eventos en este tipo de aplicaciones. Como hemos mencionado, los eventos en **Streamlit** pueden ser acciones del usuario, como hacer clic en un botón o seleccionar un elemento en una lista desplegable, que desencadenan respuestas específicas en la aplicación. Esto significa que la aplicación puede reaccionar de manera dinámica a las interacciones del usuario, ofreciendo una experiencia dinámica e interactiva al usuario.

Para implementar esta programación orientada a eventos en **Streamlit**, podemos definir funciones que se ejecuten en respuesta a eventos específicos, como el cambio de un control de usuario. Por ejemplo, cuando un usuario selecciona una opción en un menú desplegable, podemos definir una función que se ejecute en respuesta a este evento y actualice la interfaz de usuario de manera dinámica. En síntesis, así es como funciona esta programación en **Streamlit**.

Instalación, configuración del ambiente de desarrollo, y creación de aplicación simple en Streamlit

Es momento de revisar cómo instalar y configurar las herramientas necesarias para nuestro ambiente de desarrollo que nos permitirán crear cuadros de mando usando **Python** y **Streamlit**. Instalaremos el gestor de versiones de código **Git**, **GitHub**, **Python** y **Streamlit**. Cabe señalar que cada proyecto en **Python** y **Streamlit** debe ser configurado para ser usado como un repositorio de **git**, para ser guardado en una herramienta de control de versiones, y estar listo para ser desplegado en producción en una nube de cómputo.

Por su parte, **una herramienta de versionamiento de código usa el sistema de control de versiones Git diseñado por Linus Torvalds, creador del sistema operativo Linux. En ese sentido, en un sistema de gestión de versiones, los desarrolladores pueden administrar su**

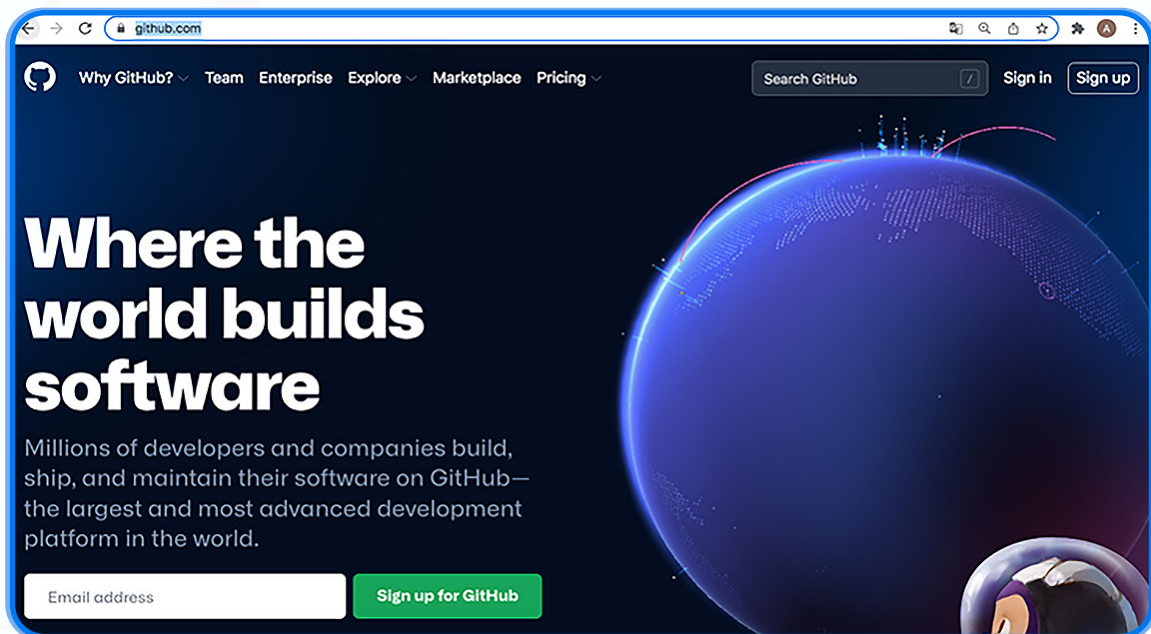
proyecto, ordenar el código de cada una de las nuevas versiones y así evitar confusiones. De ese modo, al tener copias de cada una de las versiones de su aplicación, no se perderán los estados anteriores cuando se actualiza el código.

Creación de cuenta en GitHub

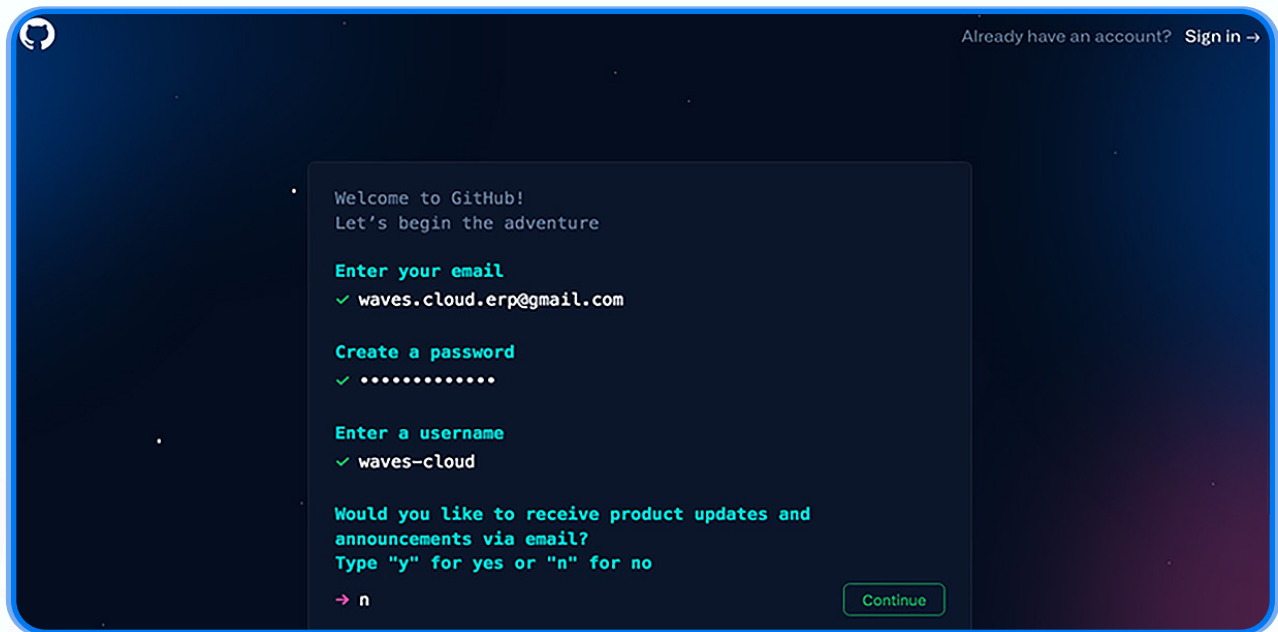
Antes de instalar **Python** y **Streamlit**, crearemos una cuenta en la plataforma de GitHub. A continuación, te mostramos cómo hacerlo en Git en la nube, sin embargo, con el paso del tiempo, la interfaz puede mostrar actualizaciones.

Paso 1. Para usar **GitHub**, primero debemos crear una cuenta, así que ingresa a: <https://github.com/>, y presiona el botón **"Sign up"** que aparece en la esquina superior derecha.

Nota: Copia y pega el enlace en tu navegador para acceder.



Paso 2. Ingresa tu correo electrónico (puede ser uno que tengas creado de Gmail, Outlook, institucional), *password* o contraseña, y nombre de usuario **git**. Este es muy importante porque todos los repositorios nuevos contienen tu nombre de usuario.



The screenshot shows the GitHub account creation interface. At the top right, there is a link "Already have an account? Sign in →". The main content area has a dark blue background with a lighter blue box containing the following text:

Welcome to GitHub!
Let's begin the adventure

Enter your email
✓ waves.cloud.erp@gmail.com

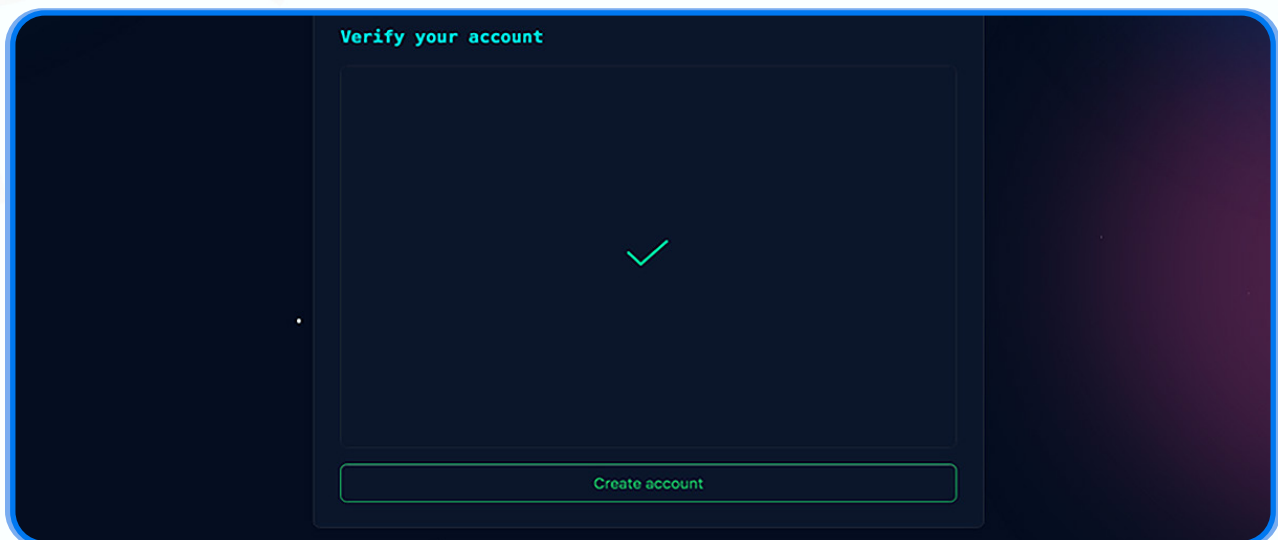
Create a password
✓

Enter a username
✓ waves-cloud

Would you like to receive product updates and announcements via email?
Type "y" for yes or "n" for no
→ n

A "Continue" button is located at the bottom right of the form.

Paso 3. Para asegurar que eres un humano y no un robot, resuelve un **Puzzle** que se genera de forma automática y aleatoria, una vez resuelto, te preguntará si deseas crear la cuenta.

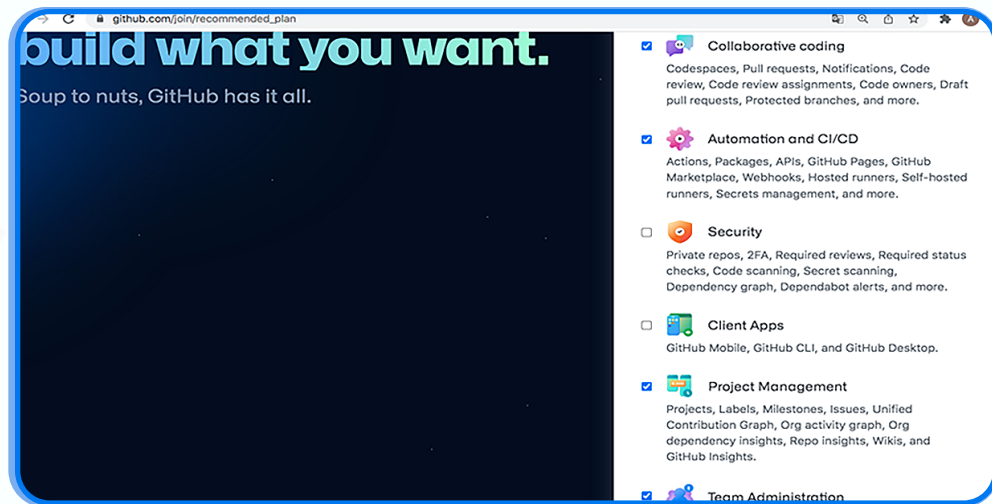


The screenshot shows the GitHub account verification screen. At the top, it says "Verify your account". Below this, there is a large dark blue box with a green checkmark in the center. At the bottom of the box, there is a "Create account" button.

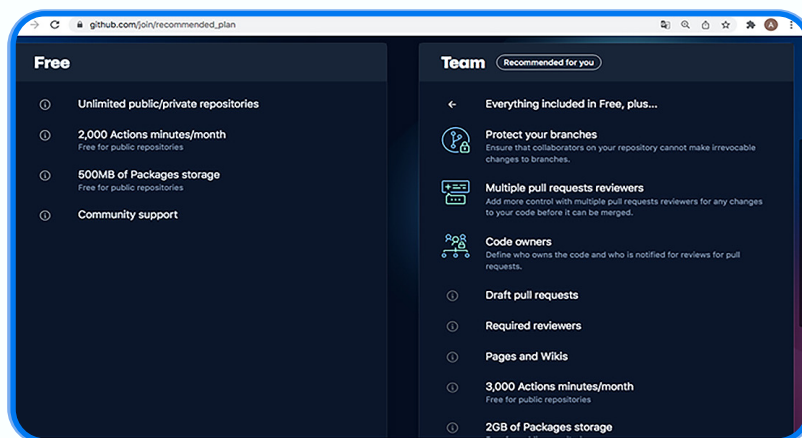
Paso 4. Revisa tu correo electrónico, ya que, debiste haber recibido un código que debes ingresar como se muestra a continuación:

Paso 5. Un aspecto importante es elegir el tamaño de tu equipo de ingenieros, considerando que los equipos de 4 pueden tener repositorios privados sin costo. Asimismo, puedes tener repositorios públicos con cualquier cantidad de ingenieros. En este caso, elige equipo de 2-5 integrantes.

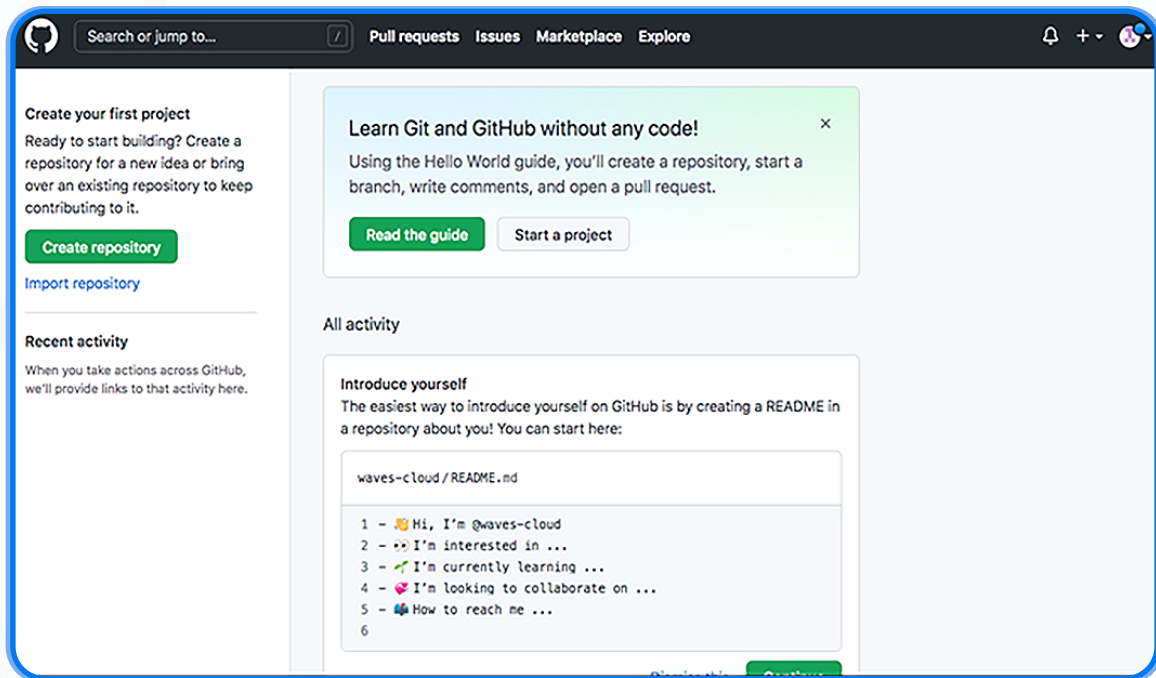
Paso 6. Selecciona las características que usarás de **GitHub**, pueden ser básicas que se relacionan con la administración de proyectos: trabajo de código colaborativo, automatización para integración y despliegue continuo, administración de proyectos, y finalmente, administración de equipos.



GitHub es gratuito con características como: repositorios públicos ilimitados sin restricción en tamaño del equipo, un repositorio público es visible para todos los usuarios de **GitHub**, cualquier persona puede clonarlo, pero no pueden escribir en él; es la opción ideal para proyectos de **open source**, con fines didácticos y/o de entrenamiento. Por otra parte, los repositorios privados con máximo 4 integrantes es nuestra mejor opción para el proyecto, porque solo es visible para los ingenieros invitados a colaborar. Por lo tanto, en esta ocasión, elegiremos la opción **Free**.



Paso 7. Para empezar con el proyecto, crea tu primer repositorio de código, para esto, da clic en el botón: **Create Repository**.



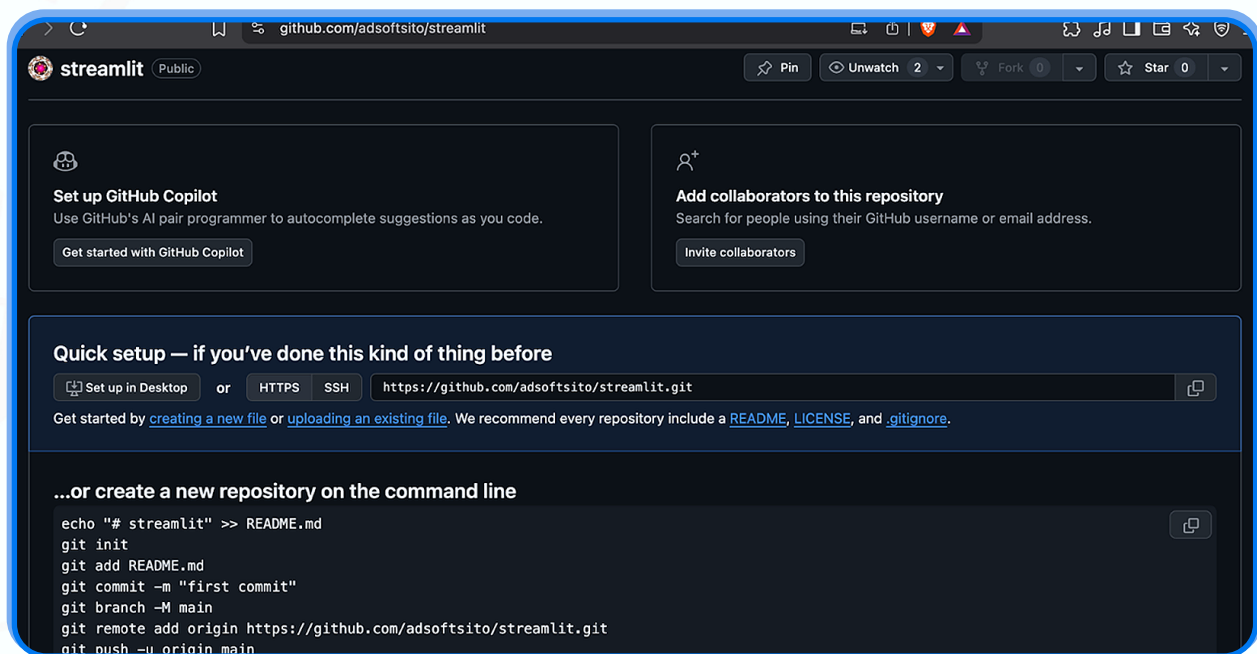
Paso 8. **Configura el repositorio**, para esto, en la pantalla de creación de repositorio, escribe el nombre **Streamlit**. Observa que el nombre está precedido por el nombre del usuario elegido en los pasos anteriores, entonces, el nombre de este repositorio sería: **<tu-usuariogit>/streamlit**, y el nombre completo: **https://github.com/<tu-usuario>/streamlit/**. Mismo que está formado por tres elementos: **https://github.com/**, nombre usuario y nombre del repositorio.

Paso 9. Selecciona si el repositorio es **Público**, visible para todo mundo, o **Privado**, visible para los miembros del equipo. Además, indica si deseas inicializar el repositorio con algunos archivos de configuración como:

- **README**: Permite documentar el propósito de nuestro repositorio, para **Angular** es importante no crear un **README**.

- **.gitignore:** Permite configurar los archivos que no deben ser subidos al repositorio, es decir, que deben ser ignorados. **Recuerda no marcar esta opción, ya que el proyecto de Angular ya contiene un .gitignore**
- **Choose a license:** Permite elegir el tipo de licencia que aplicará para nuestro proyecto (MIT license, Apache License, entre otros). **Recuerda no seleccionar ningún tipo de licencia para este proyecto.**

Paso 10. Finalmente, observa que el repositorio estará listo para usar:

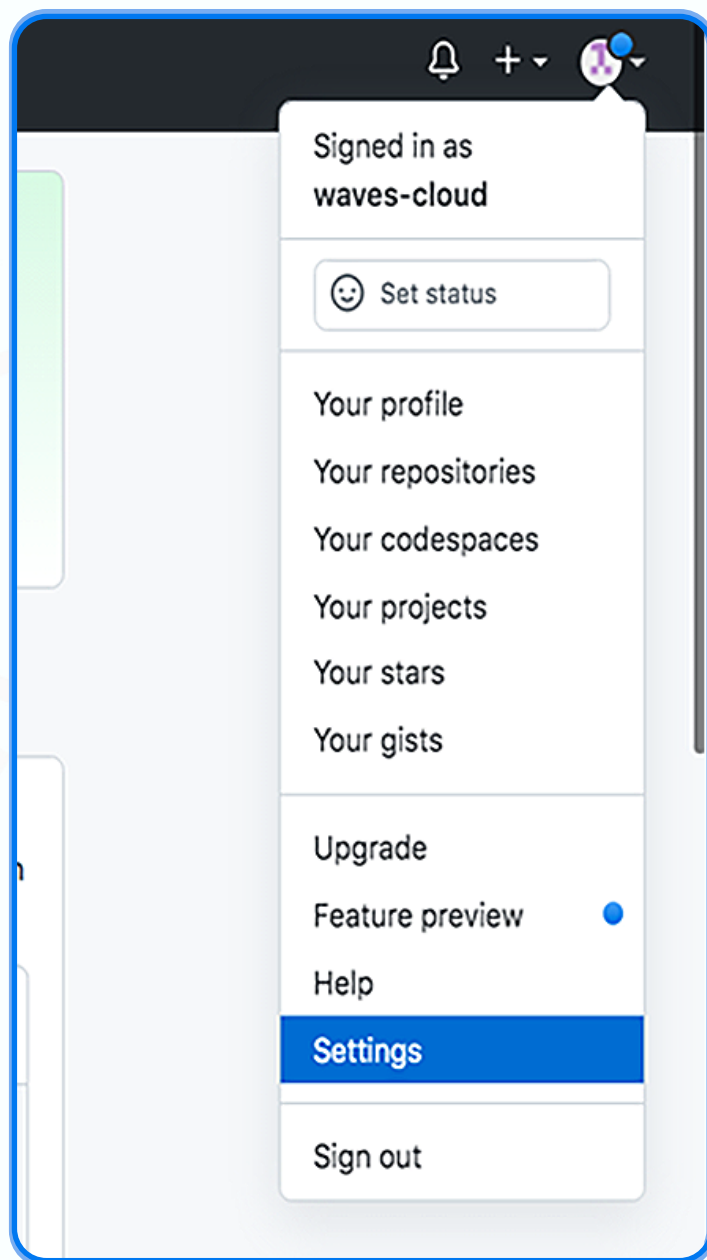


Creación de un *token* de GitHub

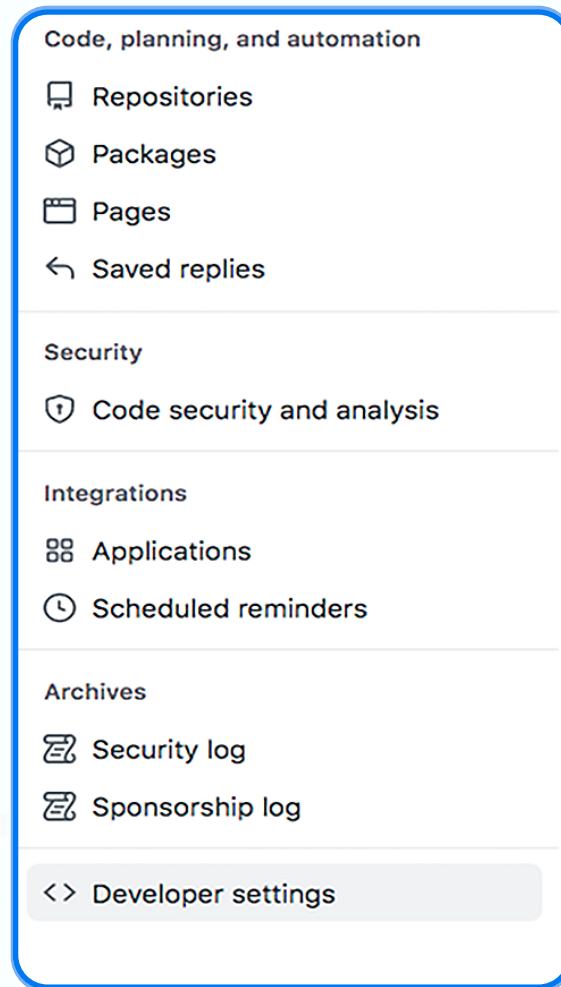
Ahora, es momento de crear **Token** para comprobar que somos los propietarios del repositorio. Para esto, debes hacer lo siguiente:

Paso 1. Ingresa nuevamente a <https://github.com> con usuario y contraseña, después, selecciona el menú de perfil en la esquina superior derecha, luego, la opción **Settings**.

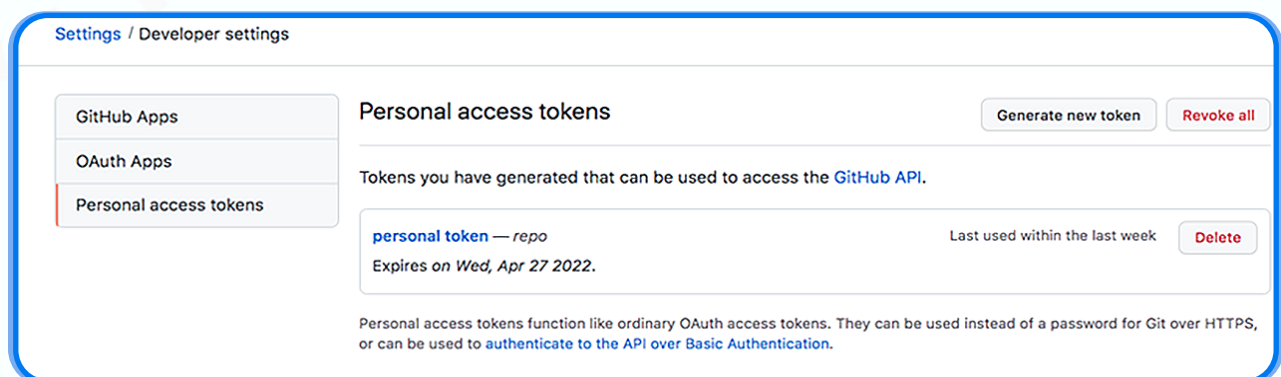
Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.



Paso 2. Ingresa a la opción **Developer Settings**.



Paso 3. Dentro de esta opción, ingresa a **Personal access tokens**.



Paso 4. Una vez dentro, debes hacer clic en **“Generate new token”**, dentro de la pantalla **New personal access token**, ingresa una nota o descripción para nuestro **token** nuevo, puedes colocar el tiempo de expiración del **token**; para este caso, será de 90 días. Finalmente, debes establecer los permisos otorgados al **token**, en este caso, selecciona los permisos relacionados con **repo**, y presiona el botón de **“Create Token”**.

GitHub Apps

OAuth Apps

Personal access tokens

New personal access token

Personal access tokens function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to authenticate to the API over Basic Authentication.

Note

personal token

What's this token for?

Expiration *

90 days The token will expire on Fri, Apr 29 2022

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo:deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows

Paso 5. Después, copia y pega el **token** en un lugar seguro, puede ser un bloc de notas o en cualquier otro editor de texto plano. Por ejemplo: **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**

Settings / Developer settings

GitHub Apps

OAuth Apps

Personal access tokens

Personal access tokens

Generate new token Revoke all

Tokens you have generated that can be used to access the GitHub API.

Make sure to copy your personal access token now. You won't be able to see it again!

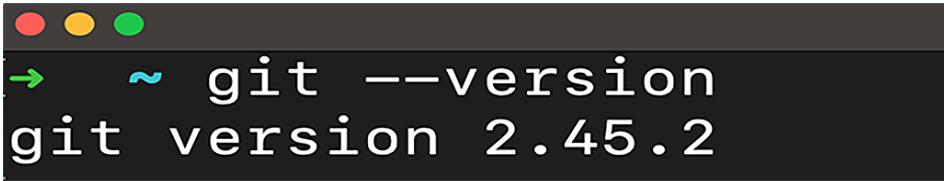
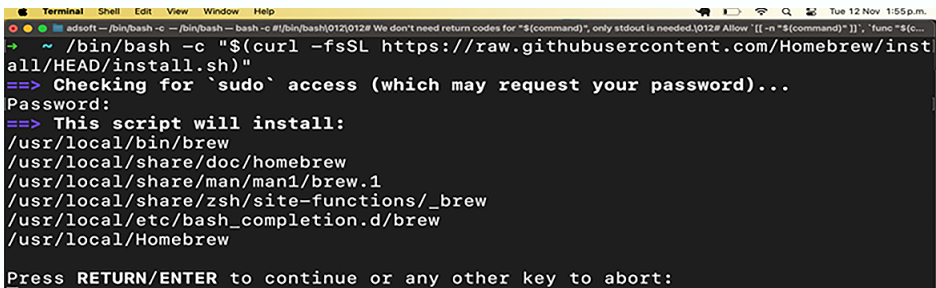
✓ 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 Delete

personal token — repo Last used within the last week Delete

Expires on Wed, Apr 27 2022.

Personal access tokens function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to authenticate to the API over Basic Authentication.

Paso 6. Ahora bien, el **token** y el nombre usuario son las credenciales para subir nuestro proyecto de **Streamlit** al repositorio de **GitHub**. Las usaremos ampliamente más adelante. Por lo pronto, debes instalar el cliente de **git** en tu computadora, así podrás subir los proyectos a la plataforma de **github.com**. Sin embargo, esto puede variar dependiendo el sistema operativo con el que cuentes, por eso te mostramos como hacerlo para tres posibles escenarios:

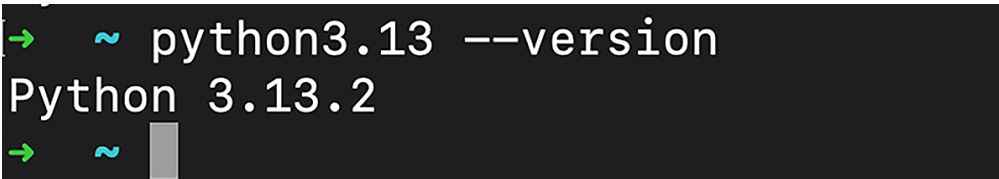
Sistema operativo	Pasos para instalar el cliente de git en una computadora
Windows	1) Descarga el instalador para 64 bits, copiando y pegando el enlace en tu navegador: https://git-scm.com/downloads/win, 2) Después, instálalo con todas las opciones predeterminadas. 3) Este paquete instalará el cliente de git, y agregará un intérprete de comandos llamado git bash, mismo que usaremos para realizar prácticas más adelante. 4) Una vez completada la instalación, busca y abre la aplicación git bash, después, prueba la instalación de git con: <code>\$ git --version</code> 
Mac	1) Instala el gestor de paquetes brew (solo en caso de no tenerlo), después, abre una terminal de Mac y ejecuta el siguiente comando: <code>\$ /bin/bash -c "\$(curl -fsSL</code> 2) Copia y pega el enlace en tu navegador para obtenerlo: https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh 

Mac	3) Una vez instalado brew, instala git con el comando: <code>\$ brew install git</code> <pre> ➤ ~ brew install git ==> Auto-updating Homebrew... Adjust how often this is run with HOMEBREW_AUTO_UPDATE_SECS or disable with HOMEBREW_NO_AUTO_UPDATE. Hide these hints with HOMEBREW_NO_ENV_HINTS (see `man brew`). </pre> 4) Finalmente, comprueba la instalación de git con: <code>\$ git --version</code>
Linux	Regularmente, los sistemas basados en Linux sean de la familia Debian (Ubuntu, Mint), o los basados en Redhat (Centos, Rocky, Fedora), ya tienen instalado git, pero puedes verificar con: <pre>\$ git --version</pre> En caso de que no exista, puedes instalarlo con: <pre>\$ sudo apt-get install git # debian, ubuntu</pre> <pre>\$ sudo yum install git # redhat, centos, fedora</pre>

Dependiendo de tu sistema operativo, y una vez instalado **git** y configurado **GitHub**, estamos listos para continuar con la instalación de **Python** de forma local en tu computadora.

Instalación de Python

La instalación de **Python** también cambia dependiendo el sistema operativo, por eso te mostramos cómo hacerlo en tres escenarios.

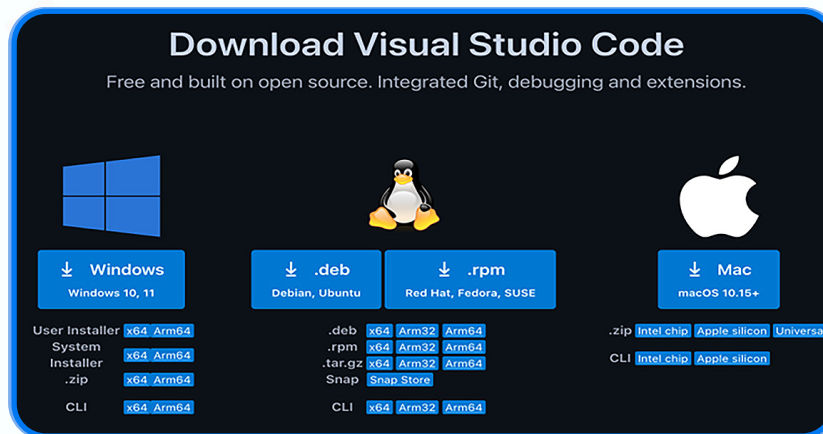
Sistema operativo	Pasos para instalar Python
Windows	1) Abre una terminal de Windows PowerShell en modo administrador, y teclea el comando: <code>\$ python</code> 2) En caso de que no se encuentre instalado, se abrirá automáticamente Windows Store, y sugerirá la instalación de la última versión de Python. Preferentemente, instala Python 3.13 o superiores.
Mac	1) Descarga el instalador de Python del sitio oficial, preferentemente, la versión 3.13.x. Copia y pega en tu navegador el enlace: https://www.python.org/downloads/macOS/ 2) Después, prueba la instalación con: <code>\$ python3.13 --version</code> 
Linux	1) Abre una terminal y ejecuta: <pre>\$ sudo add-apt-repository ppa:deadsnakes/ppa \$ sudo apt update \$ sudo apt install python3.13</pre> 2) Después, verifica la instalación con: <code>\$ python3.13 --version</code>

Instalación de Visual Studio Code

Una vez instalado **Python**, necesitas instalar el entorno de desarrollo para tus proyectos: **Visual Studio Code**. Para esto, realiza lo siguiente:

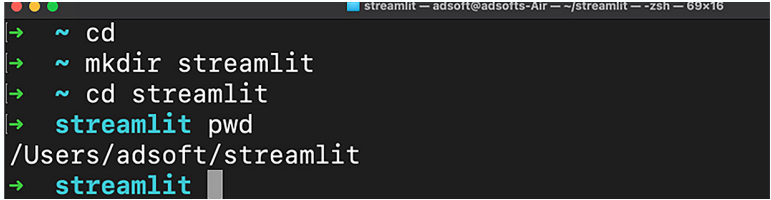
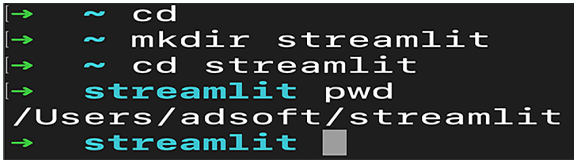
Paso 1. Descarga la versión apropiada para tu sistema operativo: **Windows, Mac o Linux**. Copia y pega el enlace en tu navegador: <https://code.visualstudio.com/download>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.

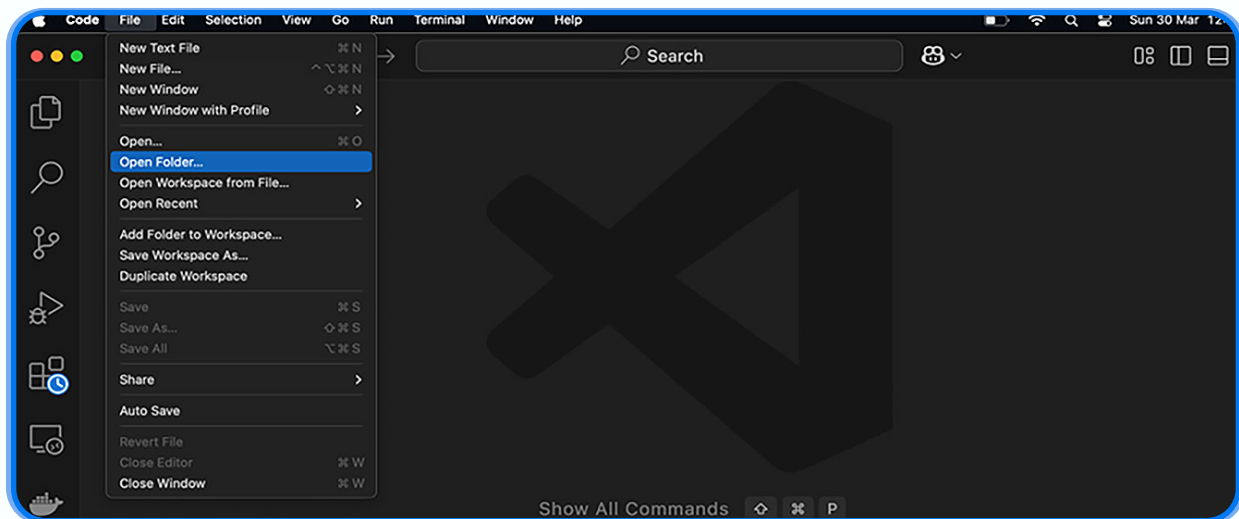


Paso 2. Descarga y ejecuta el instalador. Una vez hecho, crea un directorio de trabajo para tu proyecto desde la terminal, y antes de abrir **Visual Studio Code**, dependiendo de tu sistema operativo, realiza lo siguiente:

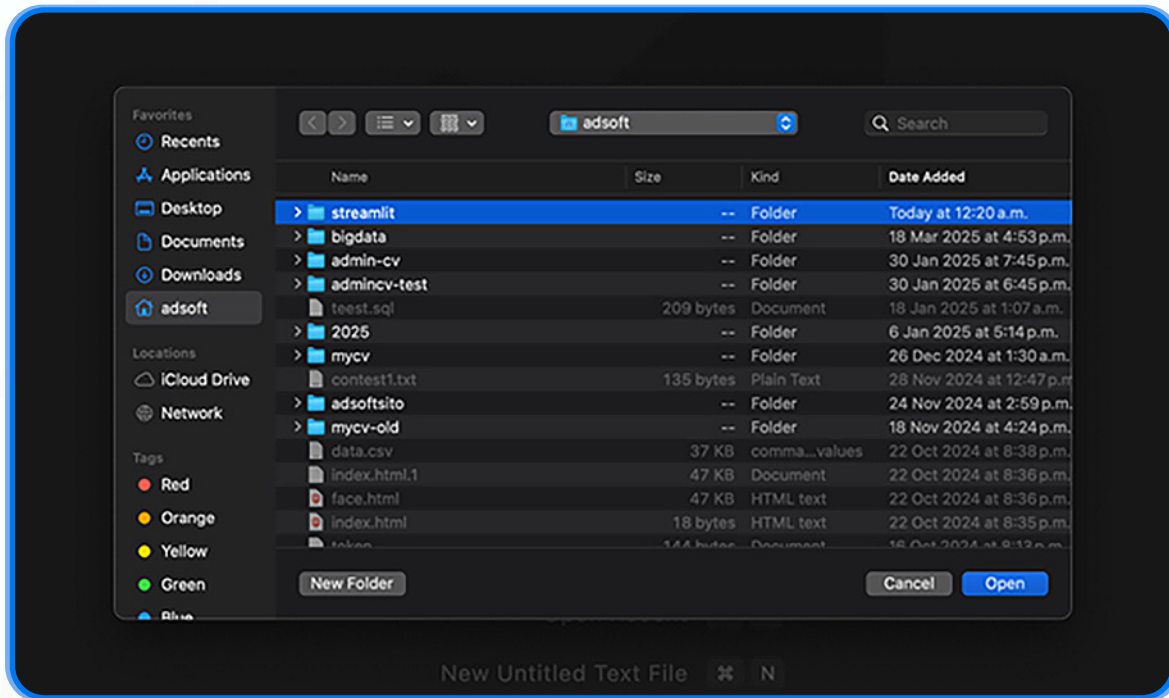
Sistema operativo	Pasos para instalar Visual Studio Code
Windows	1) Abre una terminal de git bash, no PowerShell o cmd, y ejecuta: <pre>\$ cd \$ mkdir streamlit \$ cd streamlit \$ pwd</pre>

Mac	1) Abre la aplicación terminal y ejecuta: <pre>\$ cd \$ mkdir streamlit \$ cd streamlit \$ pwd</pre> 
Linux	1) Abre una terminal de Shell y ejecuta: <pre>\$ cd \$ mkdir streamlit \$ cd streamlit \$ pwd</pre> 

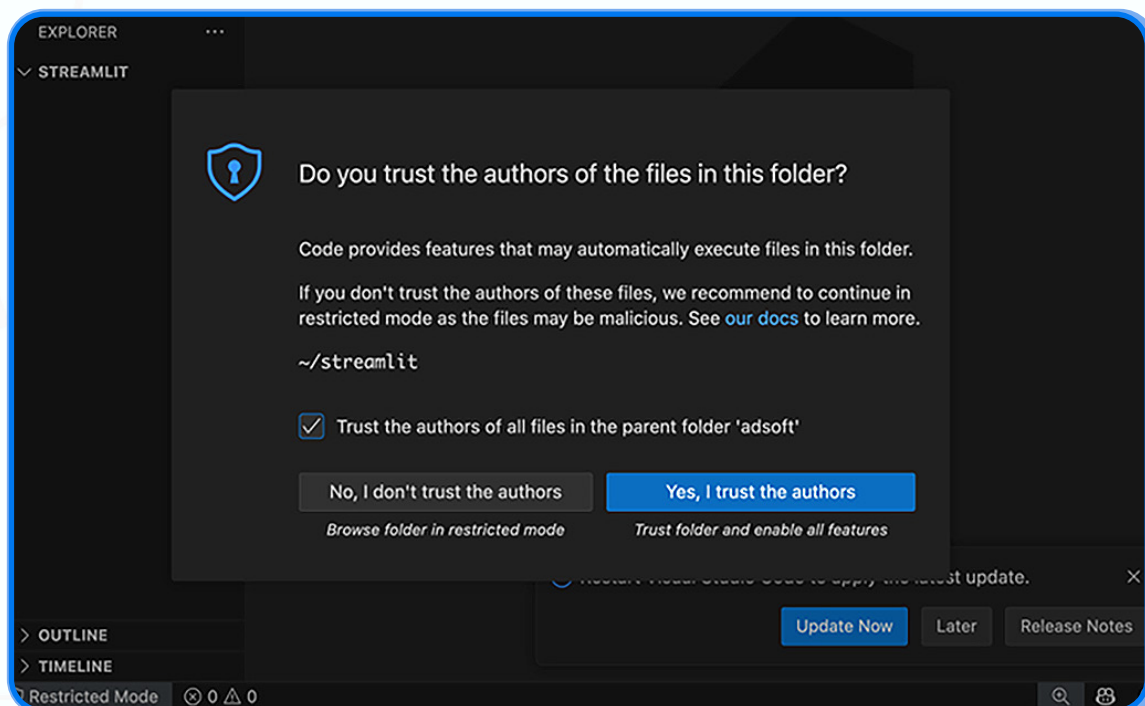
Paso 3. Una vez realizado esto, abre **Visual Studio Code**, y selecciona la opción **abrir folder**.



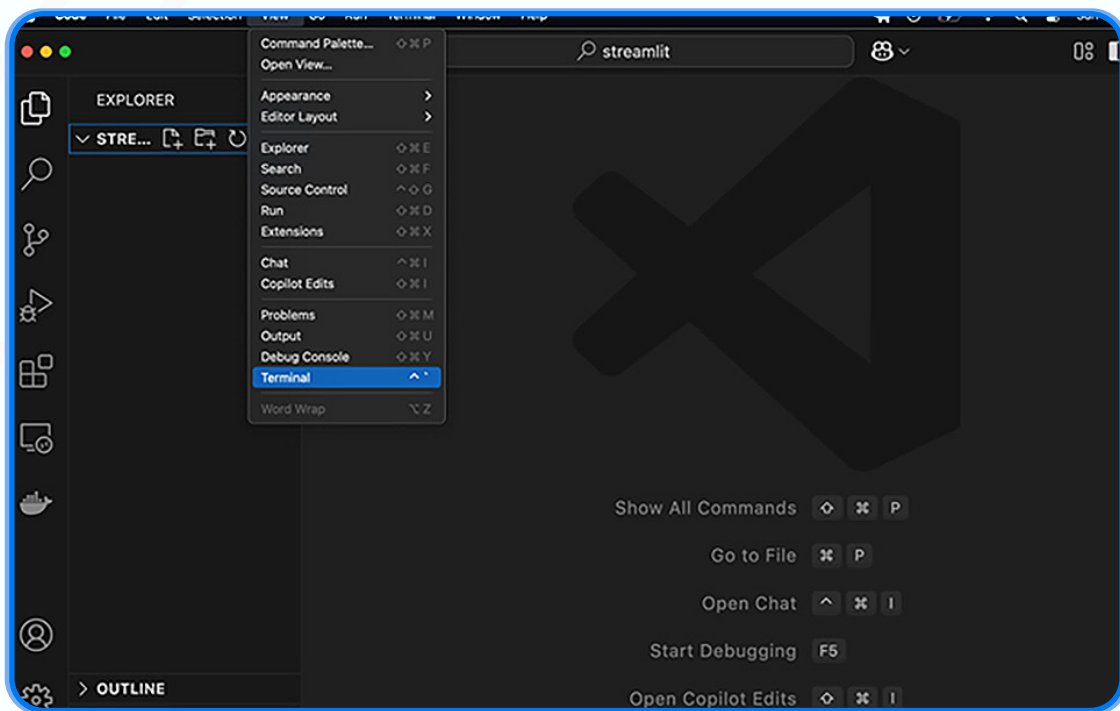
Paso 4. Selecciona el recién creado **folder** en directorio de trabajo llamado: **streamlit**.



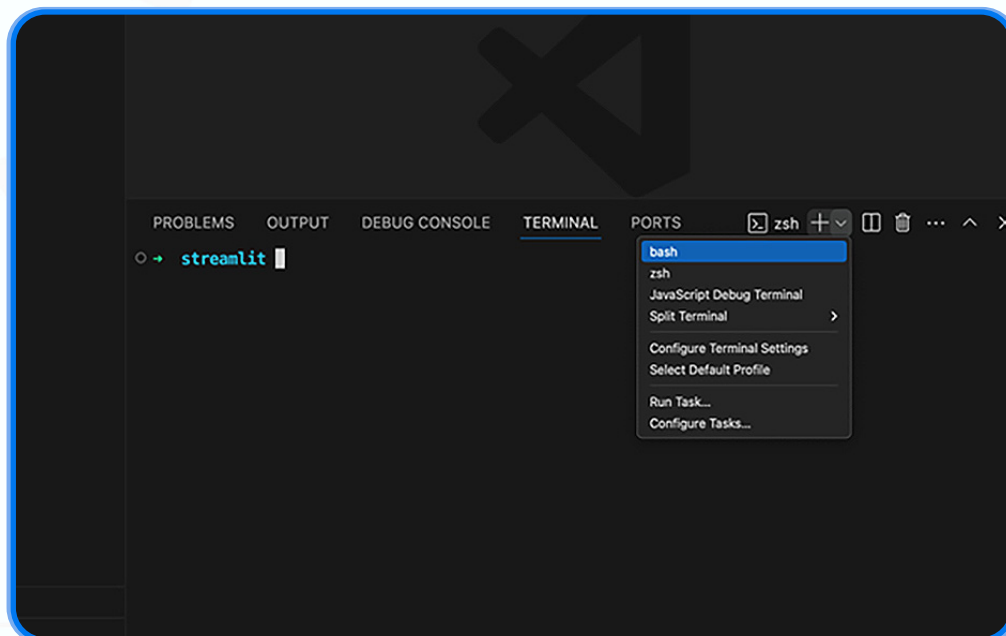
Paso 5. Acepta en el cuadro de diálogo donde confías en el contenido del **folder streamlit**.



Paso 6. Abre una terminal integrada dentro del entorno de trabajo de **Visual Studio Code**.



Paso 7. Para los usuarios de **Windows**, es importante asegurarse de crear una terminal tipo **git bash** dentro de **Visual Studio Code**, para así, tener compatibilidad con todos los comandos tipo **Linux**.



Creación de un entorno virtual

Para crear el entorno virtual a utilizar, realiza lo siguiente:

Paso 1. Desde la terminal de **Visual Studio Code**, crea un ambiente virtual de desarrollo para mantener aisladas las dependencias de tu proyecto como el *framework* **Streamlit** y todas las dependencias adicionales que requiera.

Paso 2. Para esto, utiliza la siguiente secuencia de comandos en la terminal integrada de **Visual Studio Code**.

```
$ pwd
$ python -m venv streamlit-env
$ python3.13 -m venv streamlit-env
```

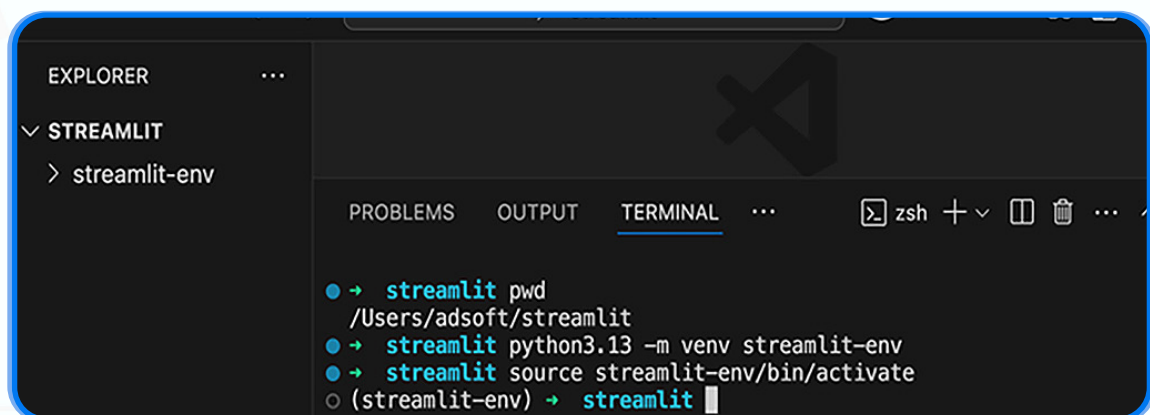
Paso 3. Activa el ambiente virtual:

Para **Linux y Mac**, utiliza:

```
$ source streamlit-env/bin/activate
```

Para **Windows** utiliza:

```
$ streamlit-env/Scripts/activate
```

A screenshot of the Visual Studio Code interface. The Explorer panel on the left shows a project named 'STREAMLIT' with a subfolder 'streamlit-env'. The Terminal panel at the bottom shows a sequence of commands executed in a 'zsh' shell: 'streamlit pwd' (output: '/Users/adsoft/streamlit'), 'streamlit python3.13 -m venv streamlit-env', and 'streamlit source streamlit-env/bin/activate'. The prompt now shows '(streamlit-env) → streamlit'.

```
EXPLORER
  ...
  ▼ STREAMLIT
    > streamlit-env

PROBLEMS  OUTPUT  TERMINAL  ...
  [zsh] + ▢ 🗑️ ...
  ● → streamlit pwd
    /Users/adsoft/streamlit
  ● → streamlit python3.13 -m venv streamlit-env
  ● → streamlit source streamlit-env/bin/activate
  ○ (streamlit-env) → streamlit
```

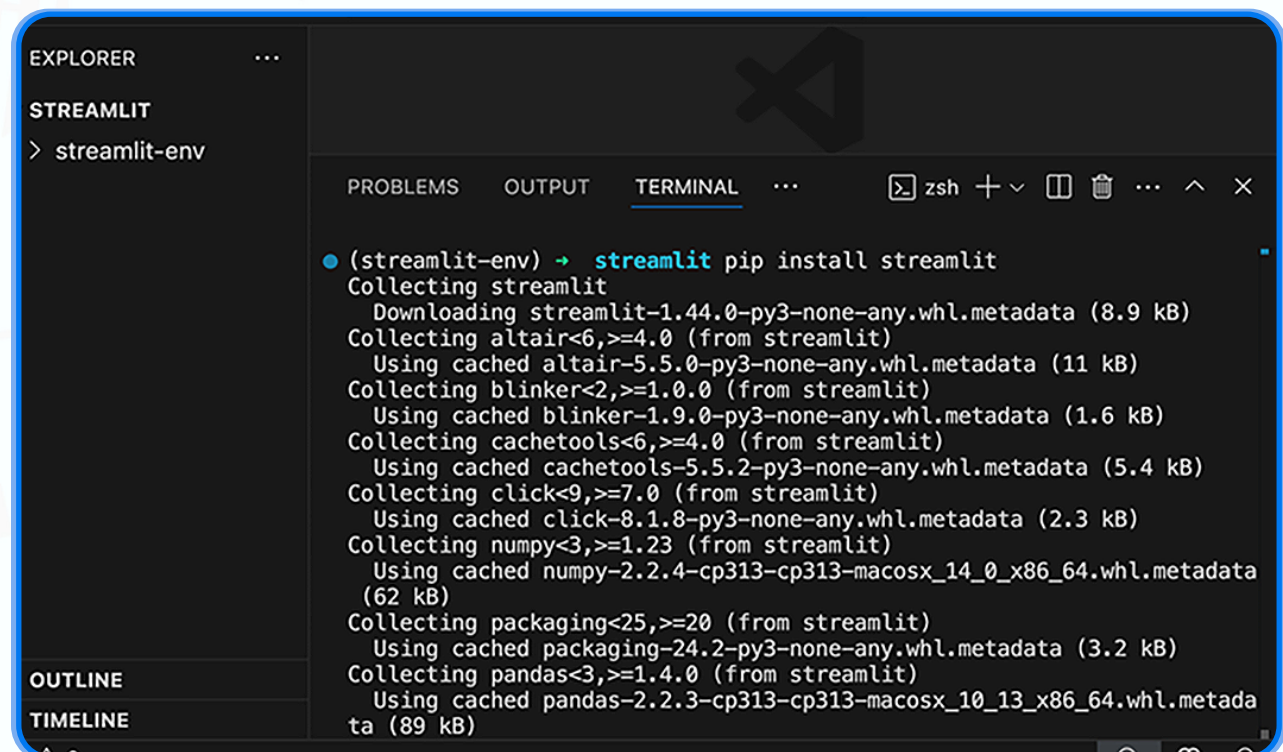
Paso 4. Observa que has activado el ambiente virtual de forma correcta cuando el **Shell** cambia a:

```
(streamlit-env) $
```

Paso 5. Una vez instalado y activado el ambiente virtual, instala el **framework Streamlit** con el comando:

```
pip install streamlit
```

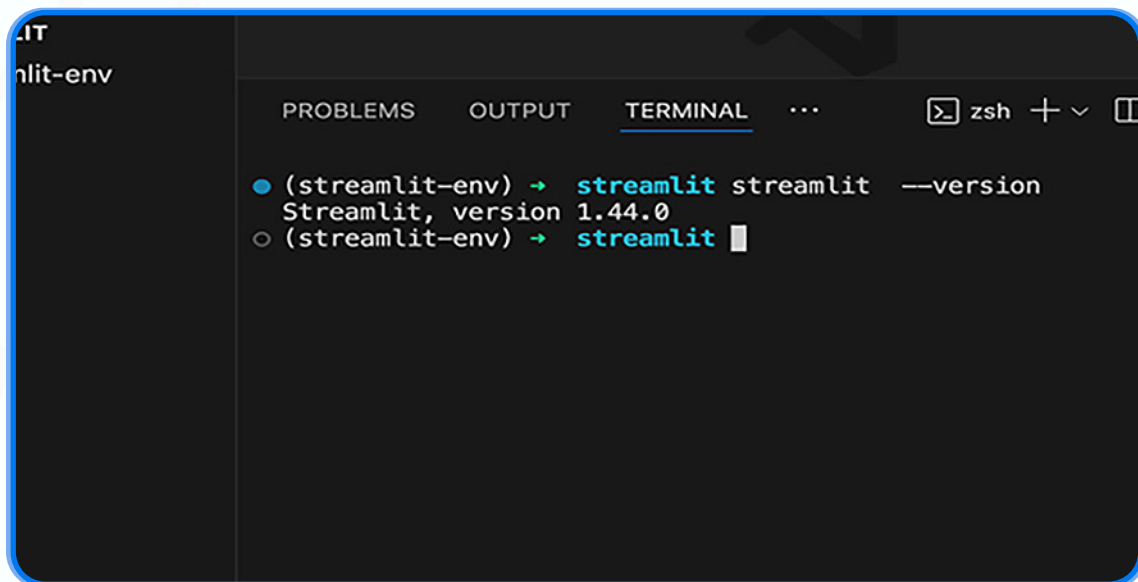
```
( streamlit-env ) $ pip install streamlit
```



The screenshot shows a Visual Studio Code interface with a terminal window open. The Explorer sidebar on the left shows a project named 'STREAMLIT' with a subdirectory 'streamlit-env'. The terminal window has tabs for 'PROBLEMS', 'OUTPUT', and 'TERMINAL', with 'TERMINAL' selected. The terminal shows the command '(streamlit-env) → streamlit pip install streamlit' and the output of the installation process, including downloading and collecting metadata for various dependencies like altair, blinker, cachetools, click, numpy, packaging, and pandas.

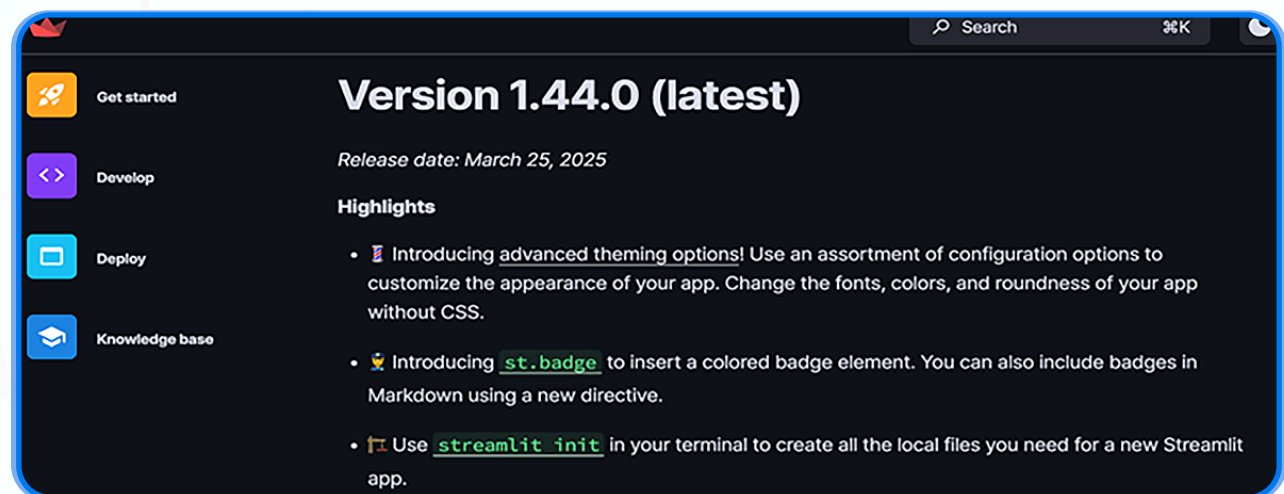
```
(streamlit-env) → streamlit pip install streamlit
Collecting streamlit
  Downloading streamlit-1.44.0-py3-none-any.whl.metadata (8.9 kB)
Collecting altair<6,>=4.0 (from streamlit)
  Using cached altair-5.5.0-py3-none-any.whl.metadata (11 kB)
Collecting blinker<2,>=1.0.0 (from streamlit)
  Using cached blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting cachetools<6,>=4.0 (from streamlit)
  Using cached cachetools-5.5.2-py3-none-any.whl.metadata (5.4 kB)
Collecting click<9,>=7.0 (from streamlit)
  Using cached click-8.1.8-py3-none-any.whl.metadata (2.3 kB)
Collecting numpy<3,>=1.23 (from streamlit)
  Using cached numpy-2.2.4-cp313-cp313-macosx_14_0_x86_64.whl.metadata (62 kB)
Collecting packaging<25,>=20 (from streamlit)
  Using cached packaging-24.2-py3-none-any.whl.metadata (3.2 kB)
Collecting pandas<3,>=1.4.0 (from streamlit)
  Using cached pandas-2.2.3-cp313-cp313-macosx_10_13_x86_64.whl.metada
ta (89 kB)
```

Paso 6. Verifica que **Streamlit** esté instalado con la versión 1.44.0 o superior.

A terminal window with a dark background and blue border. The title bar shows ".IT" and "nlit-env". The tabs are "PROBLEMS", "OUTPUT", and "TERMINAL". The terminal shows a command prompt "(streamlit-env) →" followed by the command "streamlit --version". The output is "Streamlit, version 1.44.0". The prompt is then followed by "streamlit" and a cursor.

```
.IT  
nlit-env  
  
PROBLEMS OUTPUT TERMINAL ... zsh + -  
  
● (streamlit-env) → streamlit --version  
Streamlit, version 1.44.0  
○ (streamlit-env) → streamlit
```

Nota: A la fecha de marzo de 2025, la versión 1.44.0 es la última versión estable y la que usarás en este módulo.



Paso 7. Una vez instalada correctamente la versión 1.44.0 de **Streamlit**, crea tu primera aplicación con:

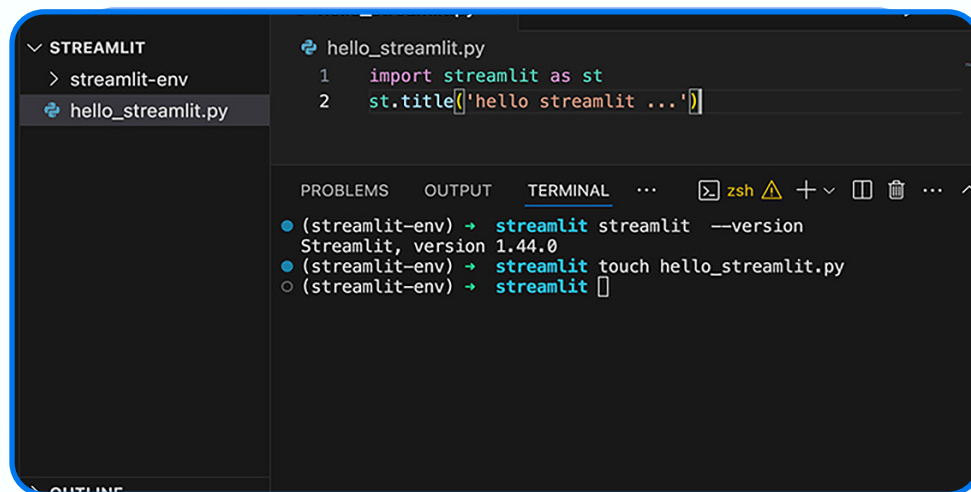
```
$ touch hello_streamlit.py
```

Paso 8. Teclea el código en el archivo recién creado en **Visual Studio Code**.

```
Import streamlit as st
```

```
st.title('hello streamlit ...')
```

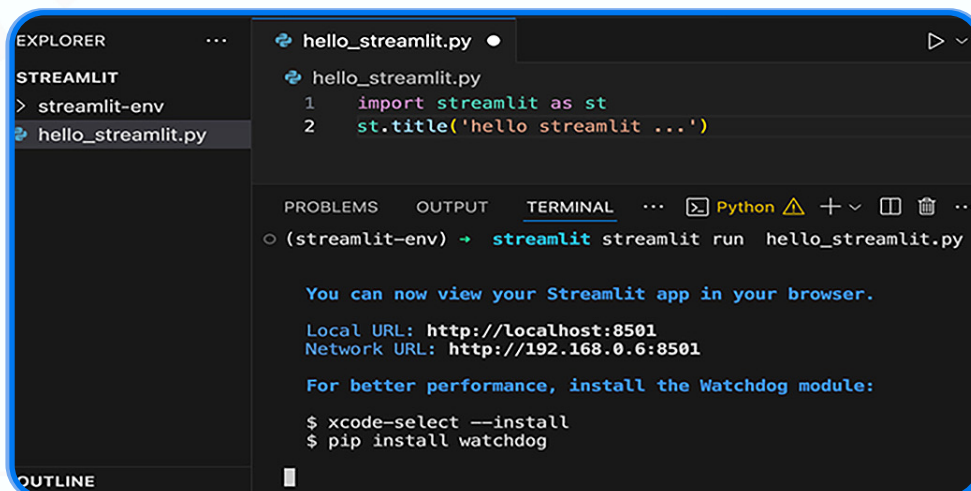
Paso 9. En este punto, asegúrate de guardar con **Ctrl-S**, **Cmd-S** o bien, usando el menú **File->Save**.



```
hello_streamlit.py
1 import streamlit as st
2 st.title('hello streamlit ...')
```

```
(streamlit-env) → streamlit run hello_streamlit.py
```

Paso 10. Ejecuta la aplicación con: `$ streamlit run hello_streamlit.py`

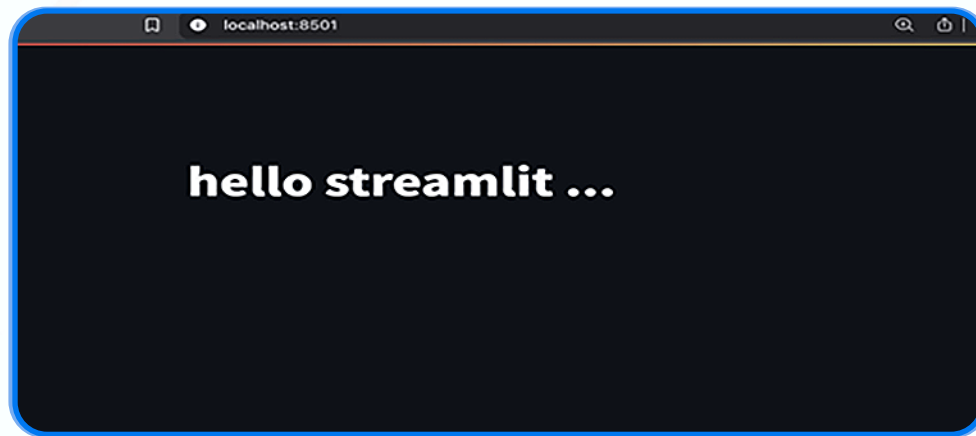


```
hello_streamlit.py
1 import streamlit as st
2 st.title('hello streamlit ...')
```

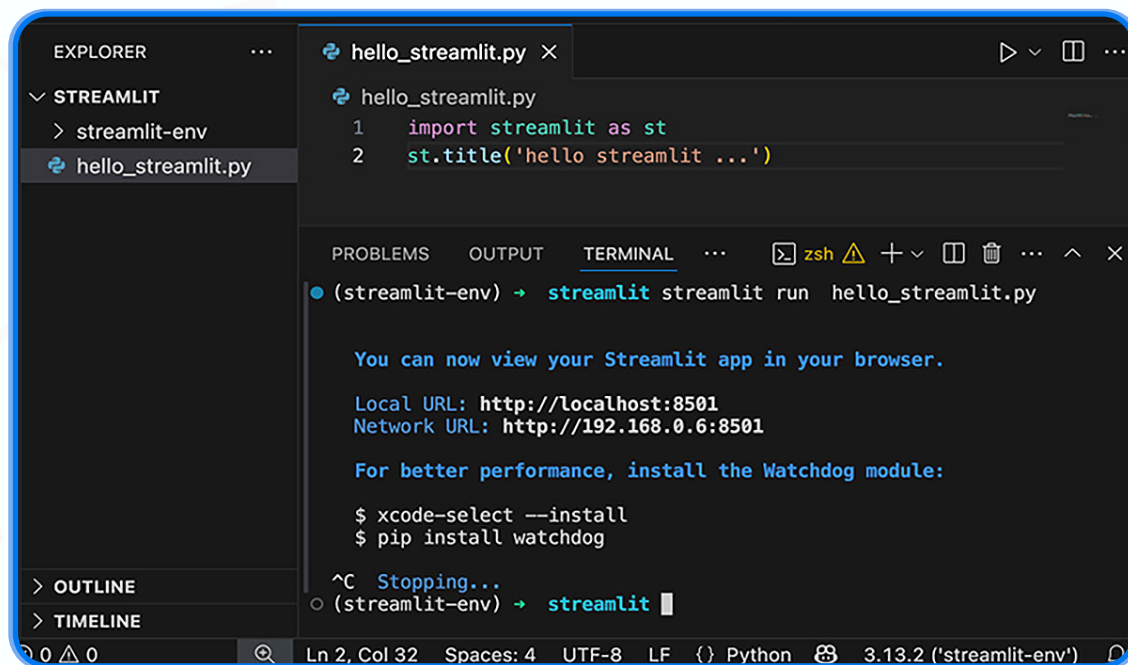
```
(streamlit-env) → streamlit run hello_streamlit.py
```

```
You can now view your Streamlit app in your browser.
Local URL: http://localhost:8501
Network URL: http://192.168.0.6:8501
For better performance, install the Watchdog module:
$ xcode-select --install
$ pip install watchdog
```

Al hacerlo, se ejecutará la aplicación en un servidor web local usando el puerto 8501, mismo que puedes visualizar en el navegador predeterminado.



Paso 11. Observa que puedes detener la ejecución de la aplicación con la combinación de teclas **Ctrl-C**.



Muy bien, hasta este punto, has instalado **git** como motor de versiones de código y **Python** como lenguaje de programación; **Streamlit** como *framework* para crear cuadros de mando, y finalmente, **Visual Studio Code** como entorno de desarrollo. Estas herramientas te permitirán trabajar con los proyectos en cada una de las lecciones del módulo.

Espacio para reflexionar

Considerando las actividades en un proyecto de esta naturaleza, que requieren trabajar en un *dashboard* o cuadro de mando, reflexiona en lo siguiente:

- ¿Cuál es la importancia de presentar información de una forma que sea más sencilla para su análisis y manipulación?
- ¿Cuáles son los componentes esenciales en un cuadro de mando?
- ¿Qué elementos debes considerar al diseñar un cuadro de mandos?
- Para construir una aplicación web en poco tiempo y con mínimas especificaciones, ¿qué plataforma utilizarías?

Como científico de datos debes contextualizar y considerar las actividades que realizas en el día a día para traducirlas en cuadros de mando que favorezcan la toma de decisiones.

Pruébate

Ejercicio de reflexión

Como científico de datos, te enfrentarás a situaciones que requieren decisiones inteligentes, por ejemplo:

- Si necesitas construir una aplicación web en un corto tiempo y con mínimas especificaciones, ¿cuáles serían tus principales opciones?, y ¿qué plataforma utilizarías?

- En caso de que necesites ejecutar un análisis de alta complejidad en tu empresa, ¿cuál es la plataforma adecuada para este fin?, ¿por qué?
- Con base en las plataformas web para el análisis de datos de sectores diferentes al tuyo, ¿cuáles son las ventajas y desventajas que puedes identificar?

Cuestionario

Instrucciones: Lee con atención cada pregunta y selecciona una respuesta.

1. En el contexto de administración de un proyecto de *software*, ¿cuál es la herramienta que permite crear versiones de código fuente?

a. Git

b. Nvm

c. Nodejs

Retroalimentación: **Git** es el motor para administrar versiones de código de los proyectos más usado en el mundo.

2. Para instalar componentes adicionales en Python debes usar ...

a. Nodejs

b. Pip

c. Streamlit

Retroalimentación: Pip es el *software* que permite instalar paquetes de Python en un proyecto.

3. ¿Cuál es el *framework* de Python para crear *dashboards* en ambientes web?

a. Streamlit

b. GitHub

c. Git

Retroalimentación: Streamlit es el *framework* de Python para crear *dashboards* interactivos para explotar *datasets*.

Ideas para llevar

- Identificaste que para la elaboración de una aplicación web, la visualización de *dashboards* permite analizar y manipular datos.
- Analizaste que en cualquier plataforma que utilices, siempre debe disponer de características como: sencillez, adaptabilidad, madurez, popularidad y soporte de Python.
- Reconociste que **git** es el motor para administrar versiones y guardar código en la nube para usarlo en las siguientes plataformas: <https://github.com>, <https://bitbucket.org> o <https://gitlab.com>.
- Estudiaste que pip permite instalar diversas componentes para Python.
- Examinaste que Streamlit permite crear aplicaciones web, sin necesidad de ser experto en HTML, CSS y JavaScript, a diferencia de tecnologías como Angular, React, Vue, JQuery, entre otras.

Bibliografía

Dayanithi. (14 de abril de 2013). *Streamlit in 3 Minutes* [Streamlit en 3 minutos]. Medium. Recuperado el 25 de abril de 2025 de: <https://medium.com/data-and-beyond/streamlit-d357935b9c>

GitHub, Inc. (2025). *What is Streamlit?* [¿Qué es Streamlit?]. GitHub. Recuperado el 25 de abril de 2025 de: <https://github.com/streamlit/streamlit>

Gurova, D. (13 de enero de 2013). *What is Streamlit: All Why's and How's Answered* [Qué es Streamlit: Todos los porqués y cómo's contestados]. Uibakery. Recuperado el 25 de abril de 2025 de: <https://uibakery.io/blog/what-is-streamlit>

Matillion. (15 de enero de 2020). *Dashboard examples: The good, the bad and the ugly* [Ejemplos de cuadros de mando: Lo bueno, lo malo y lo feo]. Matillion. Recuperado el 25 de abril de 2025 de: <https://www.matillion.com/resources/blog/dashboard-examples-the-good-the-bad-and-the-ugly>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 25 de abril de 2025 y su disponibilidad depende del autor.